Internship Hiring Test – Laravel + React Application

Objective

Develop a simple **Task Management System** using Laravel for the backend and React for the frontend. The application should allow users to create, update, delete, and list tasks.

Project Requirements

1. Backend – Laravel API

- Use **Laravel 10+** as the backend framework.
- Implement **JWT authentication** for user login and registration.
- Create a **Task model** with the following fields:
    - id (auto-increment, primary key)
    - title (string, required)
    - description (text, optional)
    - status (enum: pending, in-progress, completed, default: pending)
    - due_date (datetime, optional)
    - user_id (foreign key, links to users table)
    - created_at, updated_at (timestamps)
- Create RESTful API endpoints:
    - POST /api/register → Register a user.
    - POST /api/login → Authenticate a user and return a token.
    - GET /api/tasks → Get a list of all tasks (only for logged-in users).
    - POST /api/tasks → Create a new task (only for logged-in users).
    - PUT /api/tasks/{id} → Update a task (only the owner can update).
    - DELETE /api/tasks/{id} → Delete a task (only the owner can delete).
- Implement **validation** using Laravel Form Requests.
- Use **Eloquent ORM** for database interactions.

- Handle **authentication** with Laravel **Sanctum or JWT**.
- Write **unit tests** for at least two API endpoints.

---

2. Frontend – React

- Use **React + Vite** as the frontend framework.
- Implement **React Router** for navigation.
- Create the following pages:
    - **Login Page** → User authentication with token storage.
    - **Register Page** → User registration form.
    - **Task List Page** → Show tasks with filtering by status.
    - **Task Form Page** → Create/Edit a task.
    - **404 Page** → Display for undefined routes.
- Use **Axios** for API requests.
- Store authentication token in **localStorage**.
- Implement **state management** using **React Context** or **Redux**.
- Display validation errors properly.

Bonus Features (Optional, But Will Give Extra Points)

- **Role-based Access Control (RBAC):** Allow admin users to see all tasks, while regular users only see their own.
- **Task Filtering & Sorting:** Allow sorting by due_date and filtering by status.
- **Email Notifications:** Send an email when a task is assigned or updated.
- **Deploy the App:** Deploy the backend on **Laravel Forge/VPS/Heroku** and the frontend on **Vercel/Netlify**.

Evaluation Criteria

Criteria	Weight (%)
Code Quality & Organization	25%
Correctness & Functionality	25%
API Security & Validation	15%
Frontend UI & UX	15%
Bonus Features & Deployment	10%
Documentation (README, API Docs)	10%

Submission Requirements

- Share a **GitHub Repository** with:
    - backend/ (Laravel project)
    - frontend/ (React project)
    - README.md (Setup & usage instructions)
- If deployed, share the **live demo link**.